

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática
Escuela Profesional de Ingeniería de Software

TALLER DE CONSTRUCCIÓN DE SOFTWARE MÓVIL

Informe de Implementación — Sesión 11

NaturApp: Aplicación de Comercio Electrónico de Productos Naturales

Del Módulo a la App — Implementación Completa con
Express.js (Backend) + React Native con Expo Router (Frontend)

Lima, Perú — Junio 2026

Tabla de Contenido

1. Introducción y Objetivos
2. Tabla de Mapeo de Conceptos
3. Arquitectura del Proyecto
4. Backend — Configuración y Servidor Principal
5. Backend — Modelos de Datos (Mongoose)
6. Backend — Middleware de Autenticación
7. Backend — Rutas RESTful (Endpoints)
8. Frontend — Servicio de API (Consumo de Endpoints)
9. Frontend — Custom Hooks (Lógica de Negocio)
10. Frontend — Navegación con Expo Router
11. Frontend — Pantallas Principales
12. Frontend — Componentes Reutilizables
13. Flujo de Datos Completo
14. Tabla Resumen de Conceptos Aplicados

1. Introducción y Objetivos

Este informe presenta la implementación completa de **NaturApp**, una aplicación móvil de comercio electrónico de productos naturales inspirada en Santa Natura (<https://santanatura.com.pe/>). El proyecto aplica de manera integral cada concepto presentado en la **Sesión 11: Implementación de Software Móvil — Del Módulo a la App**.

La Sesión 11 establece tres pilares fundamentales para la implementación exitosa de aplicaciones móviles: (1) la **arquitectura modular**, que divide la aplicación en componentes independientes y cohesivos; (2) la **construcción del backend con endpoints RESTful**, que gestiona la lógica de negocio, los datos y la seguridad; y (3) la **conexión del frontend con el servidor**, que transforma una interfaz estática en una aplicación dinámica mediante el consumo de endpoints.

Objetivos del informe:

- Demostrar la arquitectura modular con módulos independientes tanto en backend como en frontend.
- Implementar un backend completo con Express.js y MongoDB, con endpoints RESTful para operaciones CRUD.
- Desarrollar un frontend React Native con Expo Router que consume los endpoints del servidor.
- Ilustrar el flujo de datos completo: desde el toque del usuario hasta la actualización de la interfaz.
- Aplicar consideraciones de performance: estados de carga (spinners), solicitudes en paralelo, paginación y pull-to-refresh.

2. Tabla de Mapeo de Conceptos

La siguiente tabla muestra cómo cada concepto de la Sesión 11 se aplica directamente en el código de NaturApp:

Concepto (Sesión 11)	Implementación en NaturApp	Archivos
Arquitectura Modular	Estructura de carpetas separada: models/, routes/, middleware/ en backend; services/, hooks/, components/ en frontend	Todo el proyecto
Modularidad	Cada módulo tiene responsabilidad única: productRoutes solo maneja productos, useCart solo maneja el carrito	Todos los archivos
Reutilización	ProductCard se usa en HomeScreen y SearchScreen; apiService se usa desde todos los hooks	ProductCard, apiService.js
Mantenimiento	Cambiar la lógica de carrito solo requiere modificar cartRoutes.js y useCart.js sin afectar otros módulos	cartRoutes.js, useCart.js
Backend con Endpoints	Express.js con rutas RESTful para productos, categorías, usuarios, pedidos y carrito	server.js, routes/*.js
RESTful APIs (CRUD)	GET = Leer, POST = Crear, PUT = Actualizar, DELETE = Eliminar, implementados en cada módulo de rutas	productRoutes.js, orderRoutes.js
Endpoints (APIs)	Cada ruta tiene URL única (/api/products, /api/orders) que responde a métodos HTTP específicos	Todas las rutas
Arquitectura Cliente-Servidor	App móvil (cliente) envía solicitudes HTTP al backend Express (servidor), que accede a MongoDB y devuelve JSON	apiService.js, server.js
Librerías de Red (fetch)	Se usa la API fetch nativa de JavaScript para todas las solicitudes HTTP desde React Native	apiService.js
Solicitud HTTP	Función request() genérica que configura método, headers (Authorization Bearer), body JSON	apiService.js
Procesamiento Backend	El servidor valida datos, ejecuta lógica de negocio (validar stock, hash de passwords), interactúa con MongoDB	productRoutes.js, orderRoutes.js
Respuesta JSON	Todas las respuestas usan formato { success: true/false, data/message } con códigos HTTP (200, 201, 400, 404, 500)	Todas las rutas
Actualización de UI	Los hooks actualizan el estado (useState/useEffect) y React Native re-renderiza la interfaz automáticamente	useProducts, useCart, useOrders
Flujo de Datos en Tiempo Real	Ciclo completo: toque del usuario, solicitud HTTP, procesamiento servidor, respuesta JSON, renderizado UI	HomeScreen, ProductDetail

Concepto (Sesión 11)	Implementación en NaturApp	Archivos
Performance (caché/spinners)	ActivityIndicator como spinner, RefreshControl para pull-to-refresh, Promise.all para solicitudes en paralelo, paginación	HomeScreen, useProducts
Ciclo Completo (módulos a app)	Integración de los 3 pilares: módulos frontend/backend, endpoints RESTful, y consumo desde la app móvil	Toda la aplicación

3. Arquitectura del Proyecto

NaturApp sigue una arquitectura modular de dos capas: un **backend Node.js con Express** y un **frontend React Native con Expo Router**. Cada capa está organizada en módulos independientes con responsabilidades bien definidas.

CONCEPTO APLICADO: Arquitectura Modular

La estructura de carpetas refleja los principios de modularidad: cada carpeta agrupa archivos por responsabilidad (models, routes, services, hooks, components). Los módulos pueden desarrollarse, probarse y mantenerse de forma independiente.

3.1. Estructura del Backend

```
naturapp-backend/
├── server.js           # Punto de entrada principal
├── package.json       # Dependencias del backend
├── models/            # Módulo de Modelos (Mongoose)
│   ├── Product.js    # Esquema de productos
│   ├── Category.js   # Esquema de categorías
│   ├── User.js       # Esquema de usuarios
│   └── Order.js      # Esquema de pedidos
├── routes/            # Módulo de Endpoints (RESTful)
│   ├── productRoutes.js # CRUD de productos
│   ├── categoryRoutes.js # CRUD de categorías
│   ├── userRoutes.js   # Auth y perfil
│   ├── orderRoutes.js  # Gestión de pedidos
│   └── cartRoutes.js   # Carrito de compras
└── middleware/        # Módulo de Middleware
    └── auth.js        # JWT autenticación/autorización
```

3.2. Estructura del Frontend

```
naturapp-mobile/
├── app/               # Expo Router (file-based routing)
│   ├── _layout.js    # Layout raíz (Stack)
│   ├── checkout.js   # Pantalla de checkout
│   ├── product/[id].js # Detalle de producto (dinámico)
│   └── auth/
│       ├── login.js  # Pantalla de login
│       └── register.js # Pantalla de registro
│   └── (tabs)/
│       ├── _layout.js # Layout de tabs
│       ├── home.js    # Pantalla principal
│       ├── search.js  # Búsqueda de productos
│       ├── cart.js    # Carrito de compras
│       ├── orders.js  # Historial de pedidos
│       └── profile.js  # Perfil de usuario
└── src/
    └── services/
        └── apiService.js # Módulo de consumo de API
```

```
| | | | | hooks/
| | | | | | useProducts.js    # Hook de productos
| | | | | | useCart.js       # Hook de carrito
| | | | | | useAuth.js       # Hook de autenticación
| | | | | | useOrders.js     # Hook de pedidos
| | | | | | components/
| | | | | | | ProductCard.js  # Tarjeta de producto
| | | | | | | CartItemRow.js  # Fila de item en carrito
| | | | | | | CategoryChips.js # Chips de categorías
| | | | | package.json
```

4. Backend — Configuración y Servidor Principal

4.1. package.json (Backend)

Configuración de dependencias del servidor Node.js:

```
{
  "name": "naturapp-backend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "nodemon server.js",
    "start": "node server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.14.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.9"
  }
}
```

4.2. server.js — Punto de Entrada

El servidor principal es el **corazón de la aplicación backend**. Aquí se conectan todos los módulos de rutas, se configura el middleware global y se establece la conexión con la base de datos.

```
import express from 'express';
import mongoose from 'mongoose';
import cors from 'cors';

// — Módulos de Rutas (Arquitectura Modular) —
import productRoutes from './routes/productRoutes.js';
import categoryRoutes from './routes/categoryRoutes.js';
import userRoutes from './routes/userRoutes.js';
import orderRoutes from './routes/orderRoutes.js';
import cartRoutes from './routes/cartRoutes.js';

const app = express();
const PORT = 9090;

// — Middleware Global —
app.use(cors());
app.use(express.json());

// — Conexión a Base de Datos —
mongoose.connect('mongodb://localhost:27017/naturapp')
```



```
.then(() => console.log('MongoDB conectado'))
.catch(err => console.error('Error de conexión:', err));

// — Registro de Endpoints (RESTful API) —
app.use('/api/products', productRoutes);
app.use('/api/categories', categoryRoutes);
app.use('/api/users', userRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/cart', cartRoutes);

// — Endpoint de Health Check —
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date() });
});

// — Middleware de Manejo de Errores —
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    success: false,
    message: err.message || 'Error interno del servidor'
  });
});

app.listen(PORT, () => {
  console.log(`Servidor NaturApp en puerto ${PORT}`);
});
```

CONCEPTO APLICADO: Backend con Endpoints + Arquitectura Modular

server.js importa cada módulo de rutas como un componente independiente (productRoutes, userRoutes, etc.) y los registra como endpoints bajo /api/. Cada app.use() crea un grupo de endpoints RESTful. El middleware global (cors, json) y el manejo de errores centralizado demuestran la separación de responsabilidades.

5. Backend — Modelos de Datos (Mongoose)

Los modelos definen la estructura de datos que se almacena en MongoDB. Cada modelo es un **módulo independiente** que encapsula la validación, los índices y los métodos específicos de cada entidad.

5.1. Product.js — Modelo de Producto

```
import mongoose from 'mongoose';

const productSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  description: { type: String, required: true },
  price: { type: Number, required: true, min: 0 },
  category: { type: mongoose.Schema.Types.ObjectId,
    ref: 'Category', required: true },
  image: { type: String, default: '' },
  stock: { type: Number, default: 0, min: 0 },
  isActive: { type: Boolean, default: true },
  tags: [{ type: String }],
  nutritionalInfo: {
    calories: { type: Number },
    protein: { type: String },
    fiber: { type: String }
  }
}, { timestamps: true });

// Índice para búsquedas eficientes
productSchema.index({ name: 'text', description: 'text' });

export default mongoose.model('Product', productSchema);
```

CONCEPTO APLICADO: Modularidad + Procesamiento Backend

El modelo Product define validaciones (required, min), índices de texto para búsqueda eficiente, y campos estructurados como nutritionalInfo. Mongoose valida automáticamente los datos antes de guardar, implementando la lógica de negocio en la capa de datos.

5.2. Category.js — Modelo de Categoría

```
import mongoose from 'mongoose';

const categorySchema = new mongoose.Schema({
  name: { type: String, required: true, unique: true },
  description: { type: String, default: '' },
  icon: { type: String, default: 'leaf' },
  isActive: { type: Boolean, default: true }
}, { timestamps: true });

export default mongoose.model('Category', categorySchema);
```

CONCEPTO APLICADO: Modularidad + Reutilización

Category es un módulo pequeño y enfocado que se reutiliza como referencia en Product (relación ObjectId). Su campo isActive permite eliminación lógica sin perder datos históricos.

5.3. User.js — Modelo de Usuario

```
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';

const userSchema = new mongoose.Schema({
  name:      { type: String, required: true },
  email:     { type: String, required: true, unique: true,
              lowercase: true },
  password:  { type: String, required: true, minlength: 6 },
  phone:     { type: String, default: '' },
  address:   {
    street:   { type: String },
    city:     { type: String },
    zipCode:  { type: String }
  },
  role:      { type: String, enum: ['customer', 'admin'],
              default: 'customer' }
}, { timestamps: true });

// Hash de contraseña antes de guardar
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// Método para comparar contraseñas
userSchema.methods.comparePassword = async function(candidate) {
  return bcrypt.compare(candidate, this.password);
};

export default mongoose.model('User', userSchema);
```

CONCEPTO APLICADO: Procesamiento Backend + Seguridad

User implementa hash de contraseñas con bcrypt en el hook pre('save'), demostrando procesamiento del servidor antes de persistir datos. El método comparePassword encapsula la lógica de verificación.

5.4. Order.js — Modelo de Pedido

```
import mongoose from 'mongoose';

const orderItemSchema = new mongoose.Schema({
```

```
product: { type: mongoose.Schema.Types.ObjectId,
          ref: 'Product', required: true },
name:    { type: String, required: true },
quantity: { type: Number, required: true, min: 1 },
price:   { type: Number, required: true }
});

const orderSchema = new mongoose.Schema({
  user:    { type: mongoose.Schema.Types.ObjectId,
            ref: 'User', required: true },
  items:   [orderItemSchema],
  total:   { type: Number, required: true },
  status:  { type: String,
            enum: ['pending', 'confirmed', 'shipped', 'delivered',
                  'cancelled'],
            default: 'pending' },
  shippingAddress: {
    street: { type: String, required: true },
    city:   { type: String, required: true },
    zipCode: { type: String, required: true }
  },
  paymentMethod: { type: String, default: 'cash' }
}, { timestamps: true });

export default mongoose.model('Order', orderSchema);
```

CONCEPTO APLICADO: Modularidad + Datos Estructurados

Order usa un subdocumento (orderItemSchema) para los items, demostrando composición de esquemas. El campo status con enum garantiza integridad de datos a nivel de modelo.

6. Backend — Middleware de Autenticación

El middleware de autenticación es un **módulo transversal** que se reutiliza en todas las rutas protegidas. Implementa JWT (JSON Web Tokens) para verificar la identidad del usuario en cada solicitud.

6.1. middleware/auth.js

```
import jwt from 'jsonwebtoken';

const JWT_SECRET = process.env.JWT_SECRET || 'naturapp_secret_key';

// Middleware de autenticación JWT
export const authenticate = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Token de acceso requerido'
    });
  }
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.userId = decoded.userId;
    req.userRole = decoded.role;
    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: 'Token inválido o expirado'
    });
  }
};

// Middleware de autorización por rol
export const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.userRole)) {
      return res.status(403).json({
        success: false,
        message: 'No tiene permisos para esta acción'
      });
    }
    next();
  };
};

// Generar token JWT
export const generateToken = (userId, role) => {
  return jwt.sign({ userId, role }, JWT_SECRET, {
    expiresIn: '7d'
  });
};
```

CONCEPTO APLICADO: Reutilización + Seguridad del Backend

authenticate y authorize son funciones middleware reutilizables que se importan en cualquier archivo de rutas. authenticate verifica el token JWT del header Authorization, y authorize verifica el rol del usuario. Esto centraliza la seguridad en un solo módulo, cumpliendo el principio de módulo con responsabilidad única.

7. Backend — Rutas RESTful (Endpoints)

Cada archivo de rutas es un **módulo endpoint** que implementa las operaciones CRUD usando métodos HTTP estándar. Los endpoints son las **puertas de entrada** al sistema backend, siguiendo el estándar RESTful.

7.1. productRoutes.js — CRUD de Productos

```
import express from 'express';
import Product from '../models/Product.js';
import { authenticate, authorize } from '../middleware/auth.js';

const router = express.Router();

// GET /api/products - Listar productos (público)
router.get('/', async (req, res) => {
  try {
    const { category, search, page = 1, limit = 20 } = req.query;
    const filter = { isActive: true };
    if (category) filter.category = category;
    if (search) filter.$text = { $search: search };

    const products = await Product.find(filter)
      .populate('category', 'name icon')
      .skip((page - 1) * limit)
      .limit(Number(limit))
      .sort({ createdAt: -1 });

    const total = await Product.countDocuments(filter);
    res.json({
      success: true,
      data: products,
      pagination: { page: Number(page), limit: Number(limit),
        total, pages: Math.ceil(total / limit) }
    });
  } catch (error) {
    res.status(500).json({ success: false,
      message: error.message });
  }
});

// GET /api/products/:id - Detalle de producto
router.get('/:id', async (req, res) => {
  try {
    const product = await Product.findById(req.params.id)
      .populate('category', 'name icon');
    if (!product) {
      return res.status(404).json({ success: false,
        message: 'Producto no encontrado' });
    }
    res.json({ success: true, data: product });
  } catch (error) {
```

```
    res.status(500).json({ success: false,
      message: error.message });
  }
});

// POST /api/products - Crear producto (admin)
router.post('/', authenticate, authorize('admin'),
  async (req, res) => {
    try {
      const product = new Product(req.body);
      await product.save();
      res.status(201).json({ success: true, data: product });
    } catch (error) {
      res.status(400).json({ success: false,
        message: error.message });
    }
  });

// PUT /api/products/:id - Actualizar producto (admin)
router.put('/:id', authenticate, authorize('admin'),
  async (req, res) => {
    try {
      const product = await Product.findByIdAndUpdate(
        req.params.id, req.body, { new: true, runValidators: true }
      );
      if (!product) {
        return res.status(404).json({ success: false,
          message: 'Producto no encontrado' });
      }
      res.json({ success: true, data: product });
    } catch (error) {
      res.status(400).json({ success: false,
        message: error.message });
    }
  });

// DELETE /api/products/:id - Eliminar producto (admin)
router.delete('/:id', authenticate, authorize('admin'),
  async (req, res) => {
    try {
      const product = await Product.findByIdAndUpdate(
        req.params.id, { isActive: false }, { new: true }
      );
      if (!product) {
        return res.status(404).json({ success: false,
          message: 'Producto no encontrado' });
      }
      res.json({ success: true,
        message: 'Producto eliminado correctamente' });
    } catch (error) {
      res.status(500).json({ success: false,
        message: error.message });
    }
  });
});
```



```
export default router;
```

CONCEPTO APLICADO: RESTful APIs + CRUD Completo

Implementa los 4 métodos HTTP del estándar REST: GET (listar y detalle), POST (crear), PUT (actualizar), DELETE (eliminar lógico). Los endpoints públicos no requieren autenticación, mientras que los de escritura exigen rol admin. La paginación y filtros en GET demuestran procesamiento eficiente del servidor.

7.2. userRoutes.js — Autenticación y Perfil

```
import express from 'express';
import User from '../models/User.js';
import { authenticate, generateToken } from '../middleware/auth.js';

const router = express.Router();

// POST /api/users/register - Registro de usuario
router.post('/register', async (req, res) => {
  try {
    const { name, email, password, phone } = req.body;
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ success: false,
        message: 'El email ya está registrado' });
    }
    const user = new User({ name, email, password, phone });
    await user.save();
    const token = generateToken(user._id, user.role);
    res.status(201).json({
      success: true,
      data: { user: { id: user._id, name, email,
        role: user.role }, token }
    });
  } catch (error) {
    res.status(400).json({ success: false,
      message: error.message });
  }
});

// POST /api/users/login - Inicio de sesión
router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user || !(await user.comparePassword(password))) {
      return res.status(401).json({ success: false,
        message: 'Credenciales inválidas' });
    }
    const token = generateToken(user._id, user.role);
    res.json({
```

```

        success: true,
        data: { user: { id: user._id, name: user.name,
                      email: user.email, role: user.role }, token }
    });
} catch (error) {
    res.status(500).json({ success: false,
        message: error.message });
}
});

// GET /api/users/profile - Perfil del usuario
router.get('/profile', authenticate, async (req, res) => {
    try {
        const user = await User.findById(req.userId)
            .select('-password');
        if (!user) {
            return res.status(404).json({ success: false,
                message: 'Usuario no encontrado' });
        }
        res.json({ success: true, data: user });
    } catch (error) {
        res.status(500).json({ success: false,
            message: error.message });
    }
});

// PUT /api/users/profile - Actualizar perfil
router.put('/profile', authenticate, async (req, res) => {
    try {
        const { name, phone, address } = req.body;
        const user = await User.findByIdAndUpdate(
            req.userId, { name, phone, address },
            { new: true, runValidators: true }
        ).select('-password');
        res.json({ success: true, data: user });
    } catch (error) {
        res.status(400).json({ success: false,
            message: error.message });
    }
});

export default router;

```

CONCEPTO APLICADO: Endpoints de Autenticación + Respuesta JSON

POST /register y POST /login crean y verifican credenciales respectivamente, devolviendo un token JWT. GET y PUT /profile usan el middleware authenticate para proteger datos del usuario. Todas las respuestas siguen el formato estándar { success, data/message } con códigos HTTP apropiados.

7.3. orderRoutes.js — Gestión de Pedidos

```
import express from 'express';
```

```
import Order from '../models/Order.js';
import Product from '../models/Product.js';
import { authenticate } from '../middleware/auth.js';

const router = express.Router();

// POST /api/orders - Crear pedido
router.post('/', authenticate, async (req, res) => {
  try {
    const { items, shippingAddress, paymentMethod } = req.body;
    // Validar stock y calcular total
    let total = 0;
    const orderItems = [];
    for (const item of items) {
      const product = await Product.findById(item.productId);
      if (!product || product.stock < item.quantity) {
        return res.status(400).json({ success: false,
          message: `Stock insuficiente: ${product?.name}` });
      }
      total += product.price * item.quantity;
      orderItems.push({
        product: product._id,
        name: product.name,
        quantity: item.quantity,
        price: product.price
      });
      // Descontar stock
      product.stock -= item.quantity;
      await product.save();
    }
    const order = new Order({
      user: req.userId, items: orderItems,
      total, shippingAddress, paymentMethod
    });
    await order.save();
    res.status(201).json({ success: true, data: order });
  } catch (error) {
    res.status(400).json({ success: false,
      message: error.message });
  }
});

// GET /api/orders - Listar pedidos del usuario
router.get('/', authenticate, async (req, res) => {
  try {
    const orders = await Order.find({ user: req.userId })
      .populate('items.product', 'name image')
      .sort({ createdAt: -1 });
    res.json({ success: true, data: orders });
  } catch (error) {
    res.status(500).json({ success: false,
      message: error.message });
  }
});
```

```
// GET /api/orders/:id - Detalle de pedido
router.get('/:id', authenticate, async (req, res) => {
  try {
    const order = await Order.findOne({
      _id: req.params.id, user: req.userId
    }).populate('items.product', 'name image price');
    if (!order) {
      return res.status(404).json({ success: false,
        message: 'Pedido no encontrado' });
    }
    res.json({ success: true, data: order });
  } catch (error) {
    res.status(500).json({ success: false,
      message: error.message });
  }
});

// PUT /api/orders/:id/cancel - Cancelar pedido
router.put('/:id/cancel', authenticate, async (req, res) => {
  try {
    const order = await Order.findOne({
      _id: req.params.id, user: req.userId
    });
    if (!order) {
      return res.status(404).json({ success: false,
        message: 'Pedido no encontrado' });
    }
    if (order.status !== 'pending') {
      return res.status(400).json({ success: false,
        message: 'Solo se pueden cancelar pedidos pendientes'
      });
    }
    // Restaurar stock
    for (const item of order.items) {
      await Product.findByIdAndUpdate(item.product,
        { $inc: { stock: item.quantity } });
    }
    order.status = 'cancelled';
    await order.save();
    res.json({ success: true, data: order });
  } catch (error) {
    res.status(500).json({ success: false,
      message: error.message });
  }
});

export default router;
```

CONCEPTO APLICADO: Procesamiento Backend + Lógica de Negocio

POST / demuestra procesamiento complejo del servidor: valida stock de cada producto, calcula el total, descuenta inventario, y crea el pedido en una sola transacción. PUT /:id/cancel restaura el stock al cancelar. Este es el endpoint más completo que ilustra toda la cadena de procesamiento backend.

7.4. cartRoutes.js — Carrito de Compras

```
import express from 'express';
import { authenticate } from '../middleware/auth.js';

const router = express.Router();

// Carrito en memoria por sesión (simplificado)
const carts = new Map();

// GET /api/cart - Obtener carrito
router.get('/', authenticate, (req, res) => {
  const cart = carts.get(req.userId) || { items: [] };
  const total = cart.items.reduce(
    (sum, item) => sum + item.price * item.quantity, 0
  );
  res.json({ success: true,
    data: { items: cart.items, total, count: cart.items.length }
  });
});

// POST /api/cart/add - Agregar al carrito
router.post('/add', authenticate, (req, res) => {
  const { productId, name, price, image, quantity = 1 } = req.body;
  if (!carts.has(req.userId)) {
    carts.set(req.userId, { items: [] });
  }
  const cart = carts.get(req.userId);
  const existing = cart.items.find(
    i => i.productId === productId);
  if (existing) {
    existing.quantity += quantity;
  } else {
    cart.items.push({ productId, name, price, image, quantity });
  }
  res.json({ success: true, data: cart });
});

// PUT /api/cart/:productId - Actualizar cantidad
router.put('/:productId', authenticate, (req, res) => {
  const cart = carts.get(req.userId);
  if (!cart) {
    return res.status(404).json({ success: false,
      message: 'Carrito vacío' });
  }
  const item = cart.items.find(
    i => i.productId === req.params.productId);
  if (!item) {
    return res.status(404).json({ success: false,
      message: 'Producto no encontrado en carrito' });
  }
  item.quantity = req.body.quantity;
});
```

```

    if (item.quantity <= 0) {
      cart.items = cart.items.filter(
        i => i.productId !== req.params.productId);
    }
    res.json({ success: true, data: cart });
  });

// DELETE /api/cart/:productId - Eliminar del carrito
router.delete('/:productId', authenticate, (req, res) => {
  const cart = carts.get(req.userId);
  if (cart) {
    cart.items = cart.items.filter(
      i => i.productId !== req.params.productId);
  }
  res.json({ success: true,
    message: 'Producto eliminado del carrito' });
});

// DELETE /api/cart - Vaciar carrito
router.delete('/', authenticate, (req, res) => {
  carts.set(req.userId, { items: [] });
  res.json({ success: true, message: 'Carrito vaciado' });
});

export default router;

```

CONCEPTO APLICADO: Endpoints CRUD + Métodos HTTP

Implementa todas las operaciones del carrito con métodos HTTP específicos: GET (obtener), POST /add (agregar), PUT /:productId (actualizar cantidad), DELETE /:productId (eliminar item), DELETE / (vaciar). Cada URL única responde de manera diferente según el método HTTP utilizado.

7.5. categoryRoutes.js — Categorías

```

import express from 'express';
import Category from '../models/Category.js';
import { authenticate, authorize } from '../middleware/auth.js';

const router = express.Router();

// GET /api/categories - Listar categorías
router.get('/', async (req, res) => {
  try {
    const categories = await Category.find({ isActive: true })
      .sort({ name: 1 });
    res.json({ success: true, data: categories });
  } catch (error) {
    res.status(500).json({ success: false,
      message: error.message });
  }
});

```

```

// POST /api/categories - Crear categoría (admin)
router.post('/', authenticate, authorize('admin'),
  async (req, res) => {
    try {
      const category = new Category(req.body);
      await category.save();
      res.status(201).json({ success: true, data: category });
    } catch (error) {
      res.status(400).json({ success: false,
        message: error.message });
    }
  });

// PUT /api/categories/:id - Actualizar categoría
router.put('/:id', authenticate, authorize('admin'),
  async (req, res) => {
    try {
      const category = await Category.findByIdAndUpdate(
        req.params.id, req.body,
        { new: true, runValidators: true }
      );
      if (!category) {
        return res.status(404).json({ success: false,
          message: 'Categoría no encontrada' });
      }
      res.json({ success: true, data: category });
    } catch (error) {
      res.status(400).json({ success: false,
        message: error.message });
    }
  });

// DELETE /api/categories/:id - Eliminar categoría
router.delete('/:id', authenticate, authorize('admin'),
  async (req, res) => {
    try {
      await Category.findByIdAndUpdate(req.params.id,
        { isActive: false });
      res.json({ success: true,
        message: 'Categoría eliminada' });
    } catch (error) {
      res.status(500).json({ success: false,
        message: error.message });
    }
  });

export default router;

```

CONCEPTO APLICADO: Modularidad + Mantenimiento

Un módulo enfocado que demuestra cómo cambios en la gestión de categorías no afectan a otros módulos. La eliminación lógica (`isActive: false`) mantiene la integridad referencial con los productos existentes.

8. Frontend — Servicio de API (Consumo de Endpoints)

El servicio de API es el **módulo central de comunicación** entre la aplicación móvil y el servidor backend. Utiliza la API **fetch** nativa de JavaScript para realizar solicitudes HTTP a los endpoints del servidor.

8.1. src/services/apiService.js

```
// src/services/apiService.js
// Módulo centralizado de comunicación con el backend

const BASE_URL = 'http://192.168.1.100:9090/api';
let authToken = null;

export const setToken = (token) => { authToken = token; };
export const clearToken = () => { authToken = null; };

// — Función genérica de solicitud HTTP —
const request = async (endpoint, options = {}) => {
  const url = `${BASE_URL}${endpoint}`;
  const headers = {
    'Content-Type': 'application/json',
    ...(authToken && { Authorization: `Bearer ${authToken}` }),
    ...options.headers,
  };

  try {
    const response = await fetch(url, {
      ...options,
      headers,
      body: options.body ? JSON.stringify(options.body) : undefined,
    });

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.message || `Error HTTP ${response.status}`);
    }
    return data;
  } catch (error) {
    if (error.name === 'TypeError') {
      throw new Error('Error de conexión con el servidor');
    }
    throw error;
  }
};

// — Módulo de Productos —
export const ProductAPI = {
  getAll: (params = {}) => {
    const query = new URLSearchParams(params).toString();
    return request(`/products?${query}`);
  }
};
```

```
    },
    getById: (id) => request(`/products/${id}`),
    search: (term) => request(`/products?search=${term}`),
  };

// — Módulo de Categorías —
export const CategoryAPI = {
  getAll: () => request('/categories'),
};

// — Módulo de Autenticación —
export const AuthAPI = {
  login: (email, password) =>
    request('/users/login', {
      method: 'POST', body: { email, password }
    }),
  register: (userData) =>
    request('/users/register', {
      method: 'POST', body: userData
    }),
  getProfile: () => request('/users/profile'),
  updateProfile: (data) =>
    request('/users/profile', {
      method: 'PUT', body: data
    }),
};

// — Módulo de Carrito —
export const CartAPI = {
  get: () => request('/cart'),
  addItem: (item) =>
    request('/cart/add', { method: 'POST', body: item }),
  updateQuantity: (productId, quantity) =>
    request(`/cart/${productId}`, {
      method: 'PUT', body: { quantity }
    }),
  removeItem: (productId) =>
    request(`/cart/${productId}`, { method: 'DELETE' }),
  clear: () => request('/cart', { method: 'DELETE' }),
};

// — Módulo de Pedidos —
export const OrderAPI = {
  create: (orderData) =>
    request('/orders', { method: 'POST', body: orderData }),
  getAll: () => request('/orders'),
  getById: (id) => request(`/orders/${id}`),
  cancel: (id) =>
    request(`/orders/${id}/cancel`, { method: 'PUT' }),
};
```

CONCEPTO APLICADO: Consumo de Endpoints + Librerías de Red + Arquitectura Cliente-Servidor

apiService.js centraliza TODA la comunicación con el backend en un solo módulo. La función request() configura automáticamente los headers (Content-Type, Authorization Bearer), serializa el body a JSON, y deserializa la respuesta. Cada sub-módulo (ProductAPI, CartAPI, etc.) expone métodos que corresponden exactamente a los endpoints del backend, creando un protocolo de comunicación estandarizado entre el cliente móvil y el servidor. Si la URL base del servidor cambia, solo se modifica este archivo.

9. Frontend — Custom Hooks (Lógica de Negocio)

Los Custom Hooks encapsulan la **lógica de negocio del frontend**, actuando como intermediarios entre el servicio de API y las pantallas. Cada hook gestiona su propio estado (loading, error, data) y expone funciones que las pantallas pueden invocar.

9.1. useProducts.js — Hook de Productos

```
// src/hooks/useProducts.js
import { useState, useEffect, useCallback } from 'react';
import { ProductAPI, CategoryAPI } from '../services/apiService';

export function useProducts() {
  const [products, setProducts] = useState([]);
  const [categories, setCategories] = useState([]);
  const [selectedCategory, setSelectedCategory] = useState(null);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [error, setError] = useState(null);
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);

  // Carga inicial de productos y categorías (paralelo)
  const loadInitialData = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const [prodRes, catRes] = await Promise.all([
        ProductAPI.getAll({ page: 1, limit: 20 }),
        CategoryAPI.getAll()
      ]);
      setProducts(prodRes.data);
      setCategories(catRes.data);
      setHasMore(prodRes.pagination.page
        < prodRes.pagination.pages);
      setPage(1);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => { loadInitialData(); }, [loadInitialData]);

  // Filtrar por categoría
  const filterByCategory = useCallback(async (categoryId) => {
    setSelectedCategory(categoryId);
    setLoading(true);
    try {
      const params = categoryId
        ? { category: categoryId, page: 1 }
        : { page: 1 };
    }
  }, []);
}
```

```

    const res = await ProductAPI.getAll(params);
    setProducts(res.data);
    setHasMore(res.pagination.page < res.pagination.pages);
    setPage(1);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
}, []);

// Buscar productos
const searchProducts = useCallback(async (term) => {
  if (!term.trim()) { loadInitialData(); return; }
  setLoading(true);
  try {
    const res = await ProductAPI.search(term);
    setProducts(res.data);
    setHasMore(false);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
}, [loadInitialData]);

// Cargar más productos (paginación)
const loadMore = useCallback(async () => {
  if (!hasMore || loading) return;
  const nextPage = page + 1;
  try {
    const params = { page: nextPage, limit: 20 };
    if (selectedCategory) params.category = selectedCategory;
    const res = await ProductAPI.getAll(params);
    setProducts(prev => [...prev, ...res.data]);
    setHasMore(res.pagination.page < res.pagination.pages);
    setPage(nextPage);
  } catch (err) {
    setError(err.message);
  }
}, [hasMore, loading, page, selectedCategory]);

// Pull-to-refresh
const refresh = useCallback(async () => {
  setRefreshing(true);
  await loadInitialData();
  setRefreshing(false);
}, [loadInitialData]);

return {
  products, categories, selectedCategory, loading,
  refreshing, error, hasMore,
  filterByCategory, searchProducts, loadMore, refresh
};

```

```
}
```

CONCEPTO APLICADO: Actualización de UI + Performance (Promise.all, Paginación)

useProducts demuestra el flujo de datos completo: carga datos iniciales con Promise.all (solicitudes en paralelo para productos y categorías), gestiona estados de loading y error, implementa paginación (loadMore) y pull-to-refresh. El uso de Promise.all reduce el tiempo de carga inicial al ejecutar dos solicitudes HTTP simultáneamente.

9.2. useCart.js — Hook de Carrito

```
// src/hooks/useCart.js
import { useState, useEffect, useCallback } from 'react';
import { CartAPI } from '../services/apiService';

export function useCart() {
  const [items, setItems] = useState([]);
  const [total, setTotal] = useState(0);
  const [count, setCount] = useState(0);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // Cargar carrito desde el servidor
  const loadCart = useCallback(async () => {
    setLoading(true);
    try {
      const res = await CartAPI.get();
      setItems(res.data.items);
      setTotal(res.data.total);
      setCount(res.data.count);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => { loadCart(); }, [loadCart]);

  // Agregar producto al carrito
  const addItem = useCallback(async (product) => {
    try {
      const res = await CartAPI.addItem({
        productId: product._id,
        name: product.name,
        price: product.price,
        image: product.image,
        quantity: 1
      });
      await loadCart(); // Recargar para sincronizar
    } catch (err) {
      setError(err.message);
    }
  }, [loadCart]);
}
```

```

        throw err;
      }
    }, [loadCart]);

    // Actualizar cantidad
    const updateQuantity = useCallback(async (productId, qty) => {
      try {
        await CartAPI.updateQuantity(productId, qty);
        await loadCart();
      } catch (err) {
        setError(err.message);
      }
    }, [loadCart]);

    // Eliminar producto
    const removeItem = useCallback(async (productId) => {
      try {
        await CartAPI.removeItem(productId);
        await loadCart();
      } catch (err) {
        setError(err.message);
      }
    }, [loadCart]);

    // Vaciar carrito
    const clearCart = useCallback(async () => {
      try {
        await CartAPI.clear();
        setItems([]);
        setTotal(0);
        setCount(0);
      } catch (err) {
        setError(err.message);
      }
    }, []);

    return {
      items, total, count, loading, error,
      addItem, updateQuantity, removeItem, clearCart, loadCart
    };
  }
}

```

CONCEPTO APLICADO: Consumo de Endpoints + Actualización de UI

useCart consume los endpoints del carrito (GET, POST, PUT, DELETE) y sincroniza el estado local con el servidor después de cada operación. El patrón 'optimistic update' se complementa con loadCart() para garantizar consistencia entre el frontend y el backend.

9.3. useAuth.js — Hook de Autenticación

```

// src/hooks/useAuth.js
import { useState, useEffect, useCallback } from 'react';

```

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import { AuthAPI, setToken, clearToken } from '../services/apiService';

export function useAuth() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Restaurar sesión al iniciar
  useEffect(() => {
    const restoreSession = async () => {
      try {
        const token = await AsyncStorage.getItem('auth_token');
        if (token) {
          setToken(token);
          const res = await AuthAPI.getProfile();
          setUser(res.data);
        }
      } catch (err) {
        await AsyncStorage.removeItem('auth_token');
        clearToken();
      } finally {
        setLoading(false);
      }
    };
    restoreSession();
  }, []);

  // Iniciar sesión
  const login = useCallback(async (email, password) => {
    setLoading(true);
    setError(null);
    try {
      const res = await AuthAPI.login(email, password);
      await AsyncStorage.setItem('auth_token', res.data.token);
      setToken(res.data.token);
      setUser(res.data.user);
      return res.data;
    } catch (err) {
      setError(err.message);
      throw err;
    } finally {
      setLoading(false);
    }
  }, []);

  // Registrarse
  const register = useCallback(async (userData) => {
    setLoading(true);
    setError(null);
    try {
      const res = await AuthAPI.register(userData);
      await AsyncStorage.setItem('auth_token', res.data.token);
      setToken(res.data.token);
    }
  }, []);
```



```

        setUser(res.data.user);
        return res.data;
    } catch (err) {
        setError(err.message);
        throw err;
    } finally {
        setLoading(false);
    }
}, []);

// Cerrar sesión
const logout = useCallback(async () => {
    await AsyncStorage.removeItem('auth_token');
    clearToken();
    setUser(null);
}, []);

return { user, loading, error, login, register, logout };
}

```

CONCEPTO APLICADO: Solicitud HTTP + Respuesta JSON + Persistencia

useAuth maneja el ciclo completo de autenticación: POST /login envía credenciales, recibe un token JWT en la respuesta JSON, lo persiste con AsyncStorage, y lo configura en apiService para solicitudes futuras. Al restaurar la sesión, verifica el token guardado con GET /profile.

9.4. useOrders.js — Hook de Pedidos

```

// src/hooks/useOrders.js
import { useState, useCallback } from 'react';
import { OrderAPI } from '../services/apiService';

export function useOrders() {
    const [orders, setOrders] = useState([]);
    const [loading, setLoading] = useState(false);
    const [error, setError] = useState(null);

    // Cargar pedidos del usuario
    const loadOrders = useCallback(async () => {
        setLoading(true);
        setError(null);
        try {
            const res = await OrderAPI.getAll();
            setOrders(res.data);
        } catch (err) {
            setError(err.message);
        } finally {
            setLoading(false);
        }
    }, []);

    // Crear nuevo pedido

```

```
const createOrder = useCallback(async (orderData) => {
  setLoading(true);
  try {
    const res = await OrderAPI.create(orderData);
    setOrders(prev => [res.data, ...prev]);
    return res.data;
  } catch (err) {
    setError(err.message);
    throw err;
  } finally {
    setLoading(false);
  }
}, []);

// Cancelar pedido
const cancelOrder = useCallback(async (orderId) => {
  try {
    const res = await OrderAPI.cancel(orderId);
    setOrders(prev => prev.map(o =>
      o._id === orderId ? res.data : o
    ));
  } catch (err) {
    setError(err.message);
    throw err;
  }
}, []);

return { orders, loading, error,
  loadOrders, createOrder, cancelOrder };
}
```

CONCEPTO APLICADO: CRUD desde Frontend + Mantenimiento

useOrders encapsula las operaciones de pedidos (crear, listar, cancelar) como un módulo independiente. Si la lógica de pedidos cambia, solo se modifica este hook sin afectar a otras partes de la aplicación, demostrando el principio de mantenimiento modular.

10. Frontend — Navegación con Expo Router

Expo Router implementa la navegación basada en archivos (file-based routing), donde la estructura de carpetas dentro de **app/** define automáticamente las rutas de la aplicación.

10.1. app/_layout.js — Layout Raíz

```
// app/_layout.js
import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack screenOptions={{
      headerStyle: { backgroundColor: '#1A5276' },
      headerTintColor: '#FFFFFF',
      headerTitleStyle: { fontWeight: 'bold' },
    }}>
      <Stack.Screen name="(tabs)" options={{
        headerShown: false
      }} />
      <Stack.Screen name="product/[id]" options={{
        title: 'Detalle del Producto',
        presentation: 'card'
      }} />
      <Stack.Screen name="auth/login" options={{
        title: 'Iniciar Sesión',
        presentation: 'modal'
      }} />
      <Stack.Screen name="auth/register" options={{
        title: 'Registrarse',
        presentation: 'modal'
      }} />
      <Stack.Screen name="checkout" options={{
        title: 'Finalizar Compra'
      }} />
    </Stack>
  );
}
```

10.2. app/(tabs)/_layout.js — Layout de Tabs

```
// app/(tabs)/_layout.js
import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';

export default function TabsLayout() {
  return (
    <Tabs screenOptions={{
      tabBarActiveTintColor: '#1A5276',
      tabBarInactiveTintColor: '#95A5A6',
      tabBarStyle: {
        backgroundColor: '#FFFFFF',

```

```

        borderTopWidth: 1,
        borderTopColor: '#ECF0F1',
        height: 60,
        paddingBottom: 8,
      },
      headerStyle: { backgroundColor: '#1A5276' },
      headerTintColor: '#FFFFFF',
    })>
    <Tabs.Screen name="home" options={{
      title: 'Inicio',
      tabBarIcon: ({ color, size }) =>
        <Ionicons name="home" size={size} color={color} />,
      headerTitle: 'NaturApp'
    }} />
    <Tabs.Screen name="search" options={{
      title: 'Buscar',
      tabBarIcon: ({ color, size }) =>
        <Ionicons name="search" size={size} color={color} />,
    }} />
    <Tabs.Screen name="cart" options={{
      title: 'Carrito',
      tabBarIcon: ({ color, size }) =>
        <Ionicons name="cart" size={size} color={color} />,
    }} />
    <Tabs.Screen name="orders" options={{
      title: 'Pedidos',
      tabBarIcon: ({ color, size }) =>
        <Ionicons name="receipt" size={size} color={color} />,
    }} />
    <Tabs.Screen name="profile" options={{
      title: 'Perfil',
      tabBarIcon: ({ color, size }) =>
        <Ionicons name="person" size={size} color={color} />,
    }} />
  </Tabs>
);
}

```

CONCEPTO APLICADO: Arquitectura Modular en Navegación

Los layouts de Expo Router organizan la navegación como módulos: `_layout.js` raíz define el Stack global (incluye rutas modales y de detalle), mientras `(tabs)/_layout.js` configura el Tab Navigator con 5 pestañas. Cada pantalla es un módulo independiente que se puede desarrollar y probar por separado.

11. Frontend — Pantallas Principales

Las pantallas son los componentes que el usuario ve e interactúa directamente. Cada pantalla es un **módulo independiente** que consume hooks para obtener datos y realizar acciones.

11.1. home.js — Pantalla Principal

```
// app/(tabs)/home.js
import { View, FlatList, ActivityIndicator, StyleSheet,
  RefreshControl, Text } from 'react-native';
import { useProducts } from '../../src/hooks/useProducts';
import { useCart } from '../../src/hooks/useCart';
import ProductCard from '../../src/components/ProductCard';
import CategoryChips from '../../src/components/CategoryChips';

export default function HomeScreen() {
  const {
    products, categories, selectedCategory,
    loading, refreshing, hasMore,
    filterByCategory, loadMore, refresh
  } = useProducts();
  const { addItem } = useCart();

  // Spinner de carga inicial
  if (loading && products.length === 0) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#1A5276" />
        <Text style={styles.loadingText}>
          Cargando productos...
        </Text>
      </View>
    );
  }

  return (
    <View style={styles.container}>
      { /* Filtros de categoría */ }
      <CategoryChips
        categories={categories}
        selected={selectedCategory}
        onSelect={filterByCategory}
      />

      { /* Lista de productos con paginación */ }
      <FlatList
        data={products}
        keyExtractor={({ item }) => item._id}
        numColumns={2}
        contentContainerStyle={styles.list}
        renderItem={({ item }) => (
          <ProductCard
            product={item}

```

```

        onAddToCart={() => addItem(item)}
      />
    )}
    refreshControl={
      <RefreshControl refreshing={refreshing}
        onRefresh={refresh} colors={['#1A5276']} />
    }
    onEndReached={loadMore}
    onEndReachedThreshold={0.5}
    ListFooterComponent={
      hasMore ? (
        <ActivityIndicator style={styles.footer}
          color="#1A5276" />
      ) : null
    }
  }
  ListEmptyComponent={
    <Text style={styles.empty}>
      No se encontraron productos
    </Text>
  }
/ >
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F8F9FA' },
  center: { flex: 1, justifyContent: 'center',
    alignItems: 'center' },
  loadingText: { marginTop: 12, color: '#7F8C8D',
    fontSize: 14 },
  list: { padding: 8 },
  footer: { paddingVertical: 20 },
  empty: { textAlign: 'center', marginTop: 40,
    color: '#95A5A6', fontSize: 16 }
});

```

CONCEPTO APLICADO: Flujo de Datos en Tiempo Real + Performance

HomeScreen ilustra el flujo completo de datos: al cargar, useProducts hace solicitudes HTTP al backend, el servidor procesa y devuelve JSON, y React Native renderiza la lista. ActivityIndicator muestra un spinner mientras se carga (0-1000ms del flujo). RefreshControl permite pull-to-refresh, y onEndReached implementa paginación infinita para optimizar la performance.

11.2. product/[id].js — Detalle de Producto

```

// app/product/[id].js
import { useState, useEffect } from 'react';
import { View, Text, Image, ScrollView, TouchableOpacity,
  ActivityIndicator, StyleSheet, Alert } from 'react-native';
import { useLocalSearchParams } from 'expo-router';
import { ProductAPI } from '../../src/services/apiService';
import { useCart } from '../../src/hooks/useCart';

```

```

import { Ionicons } from '@expo/vector-icons';

export default function ProductDetailScreen() {
  const { id } = useLocalSearchParams();
  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);
  const { addItem } = useCart();

  // Consumo del endpoint GET /products/:id
  useEffect(() => {
    const fetchProduct = async () => {
      try {
        const res = await ProductAPI.getById(id);
        setProduct(res.data);
      } catch (err) {
        Alert.alert('Error', err.message);
      } finally {
        setLoading(false);
      }
    };
    fetchProduct();
  }, [id]);

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#1A5276" />
      </View>
    );
  }

  if (!product) {
    return (
      <View style={styles.center}>
        <Ionicons name="alert-circle" size={48}
          color="#E74C3C" />
        <Text style={styles.errorText}>
          Producto no encontrado
        </Text>
      </View>
    );
  }

  const handleAddToCart = async () => {
    try {
      await addItem(product);
      Alert.alert('Agregado',
        `${product.name} agregado al carrito`);
    } catch (err) {
      Alert.alert('Error', err.message);
    }
  };

  return (

```

```

<ScrollView style={styles.container}>
  <Image source={{ uri: product.image }}
    style={styles.image} />
  <View style={styles.content}>
    <Text style={styles.category}>
      {product.category?.name}
    </Text>
    <Text style={styles.name}>{product.name}</Text>
    <Text style={styles.price}>
      S/ {product.price.toFixed(2)}
    </Text>
    <Text style={styles.description}>
      {product.description}
    </Text>

    {product.nutritionalInfo && (
      <View style={styles.nutritionBox}>
        <Text style={styles.nutritionTitle}>
          Información Nutricional
        </Text>
        <Text>Calorías: {product.nutritionalInfo.calories}</Text>
        <Text>Proteína: {product.nutritionalInfo.protein}</Text>
        <Text>Fibra: {product.nutritionalInfo.fiber}</Text>
      </View>
    )}

    <View style={styles.stockRow}>
      <Icons name={product.stock > 0
        ? "checkmark-circle" : "close-circle"}
        size={20}
        color={product.stock > 0 ? "#148F77" : "#E74C3C"} />
      <Text style={[styles.stock,
        { color: product.stock > 0 ? "#148F77" : "#E74C3C" }]}>
        {product.stock > 0
          ? ` ${product.stock} disponibles`
          : 'Agotado'}
      </Text>
    </View>

    <TouchableOpacity style={[styles.addButton,
      !product.stock && styles.disabledButton]}
      onPress={handleAddToCart}
      disabled={!product.stock}>
      <Icons name="cart" size={22} color="#FFF" />
      <Text style={styles.addButtonText}>
        Agregar al Carrito
      </Text>
    </TouchableOpacity>
  </View>
</ScrollView>
);
}

const styles = StyleSheet.create({

```



```

container: { flex: 1, backgroundColor: '#FFF' },
center: { flex: 1, justifyContent: 'center',
  alignItems: 'center' },
image: { width: '100%', height: 300 },
content: { padding: 20 },
category: { fontSize: 13, color: '#148F77',
  textTransform: 'uppercase', fontWeight: '600' },
name: { fontSize: 24, fontWeight: 'bold', color: '#1C2833',
  marginTop: 8 },
price: { fontSize: 28, fontWeight: 'bold', color: '#1A5276',
  marginTop: 8 },
description: { fontSize: 15, color: '#566573', marginTop: 16,
  lineHeight: 22 },
nutritionBox: { marginTop: 16, padding: 12,
  backgroundColor: '#E8F8F5', borderRadius: 8 },
nutritionTitle: { fontWeight: 'bold', marginBottom: 8,
  color: '#148F77' },
stockRow: { flexDirection: 'row', alignItems: 'center',
  marginTop: 16 },
stock: { marginLeft: 8, fontSize: 14 },
addButton: { flexDirection: 'row', backgroundColor: '#148F77',
  padding: 16, borderRadius: 12, alignItems: 'center',
  justifyContent: 'center', marginTop: 24 },
disabledButton: { backgroundColor: '#BDC3C7' },
addButtonText: { color: '#FFF', fontSize: 18,
  fontWeight: 'bold', marginLeft: 10 },
errorText: { marginTop: 12, color: '#E74C3C', fontSize: 16 }
});

```

CONCEPTO APLICADO: Consumo de Endpoint Individual + Actualización UI

La ruta dinámica [id].js usa useLocalSearchParams() de Expo Router para obtener el ID del producto. useEffect dispara GET /products/:id al montar el componente, y la respuesta JSON actualiza la UI mostrando imagen, nombre, precio, descripción e info nutricional. El botón 'Agregar al Carrito' dispara POST /cart/add.

11.3. cart.js — Pantalla de Carrito

```

// app/(tabs)/cart.js
import { View, Text, FlatList, TouchableOpacity,
  ActivityIndicator, StyleSheet, Alert } from 'react-native';
import { useRouter } from 'expo-router';
import { useCart } from '../../src/hooks/useCart';
import CartItemRow from '../../src/components/CartItemRow';
import { Icons } from '@expo/vector-icons';

export default function CartScreen() {
  const { items, total, loading, updateQuantity,
    removeItem, clearCart } = useCart();
  const router = useRouter();

  if (loading) {
    return (

```

```

        <View style={styles.center}>
          <ActivityIndicator size="large" color="#1A5276" />
        </View>
      );
    }

    if (items.length === 0) {
      return (
        <View style={styles.center}>
          <Icons name="cart-outline" size={64}
            color="#BDC3C7" />
          <Text style={styles.emptyText}>
            Tu carrito está vacío
          </Text>
        </View>
      );
    }

    const handleCheckout = () => {
      router.push('/checkout');
    };

    return (
      <View style={styles.container}>
        <FlatList
          data={items}
          keyExtractor={(item) => item.productId}
          renderItem={({ item }) => (
            <CartItemRow
              item={item}
              onUpdateQty={qty =>
                updateQuantity(item.productId, qty)}
              onRemove={() => removeItem(item.productId)}
            />
          )}
          contentContainerStyle={styles.list}
        />

        { /* Resumen del carrito */ }
        <View style={styles.summary}>
          <View style={styles.totalRow}>
            <Text style={styles.totalLabel}>Total:</Text>
            <Text style={styles.totalAmount}>
              S/ {total.toFixed(2)}
            </Text>
          </View>
          <TouchableOpacity style={styles.checkoutBtn}
            onPress={handleCheckout}>
            <Text style={styles.checkoutText}>
              Proceder al Pago
            </Text>
            <Icons name="arrow-forward" size={20}
              color="#FFF" />
          </TouchableOpacity>

```

```

    <TouchableOpacity style={styles.clearBtn}
      onPress={() => {
        Alert.alert('Vaciar carrito',
          '¿Estás seguro?',
          [{ text: 'Cancelar' },
            { text: 'Sí', onPress: clearCart }]);
      }}>
    <Text style={styles.clearText}>Vaciar carrito</Text>
  </TouchableOpacity>
</View>
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F8F9FA' },
  center: { flex: 1, justifyContent: 'center',
    alignItems: 'center' },
  emptyText: { marginTop: 16, fontSize: 16, color: '#95A5A6' },
  list: { padding: 12 },
  summary: { padding: 16, backgroundColor: '#FFF',
    borderTopWidth: 1, borderTopColor: '#ECF0F1' },
  totalRow: { flexDirection: 'row',
    justifyContent: 'space-between',
    marginBottom: 16 },
  totalLabel: { fontSize: 18, fontWeight: '600',
    color: '#2C3E50' },
  totalAmount: { fontSize: 22, fontWeight: 'bold',
    color: '#1A5276' },
  checkoutBtn: { flexDirection: 'row', backgroundColor: '#148F77',
    padding: 16, borderRadius: 12,
    justifyContent: 'center', alignItems: 'center' },
  checkoutText: { color: '#FFF', fontSize: 17,
    fontWeight: 'bold', marginRight: 8 },
  clearBtn: { marginTop: 10, padding: 10,
    alignItems: 'center' },
  clearText: { color: '#E74C3C', fontSize: 14 }
});

```

CONCEPTO APLICADO: CRUD desde el Frontend + Solicitudes HTTP

CartScreen demuestra las 4 operaciones CRUD del carrito: Read (GET al cargar), Update (PUT al cambiar cantidad), Delete (DELETE al eliminar ítem o vaciar). Cada acción del usuario genera una solicitud HTTP al backend, que procesa la operación y devuelve la respuesta JSON para actualizar la UI.

11.4. search.js — Búsqueda de Productos

```

// app/(tabs)/search.js
import { useState, useCallback } from 'react';
import { View, TextInput, FlatList, ActivityIndicator,
  StyleSheet, Text } from 'react-native';
import { ProductAPI } from '../../src/services/apiService';

```

```

import ProductCard from '../src/components/ProductCard';
import { useCart } from '../src/hooks/useCart';
import { Ionicons } from '@expo/vector-icons';

export default function SearchScreen() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const [searched, setSearched] = useState(false);
  const { addItem } = useCart();

  // Búsqueda con debounce manual
  const handleSearch = useCallback(async (text) => {
    setQuery(text);
    if (text.length < 2) { setResults([]); return; }
    setLoading(true);
    setSearched(true);
    try {
      const res = await ProductAPI.search(text);
      setResults(res.data);
    } catch (err) {
      console.error('Error en búsqueda:', err.message);
    } finally {
      setLoading(false);
    }
  }, []);

  return (
    <View style={styles.container}>
      <View style={styles.searchBar}>
        <Ionicons name="search" size={20} color="#95A5A6" />
        <TextInput style={styles.input}
          placeholder="Buscar productos naturales..."
          value={query}
          onChangeText={handleSearch}
          placeholderTextColor="#95A5A6"
        />
      </View>

      {loading ? (
        <ActivityIndicator style={styles.loader}
          size="large" color="#1A5276" />
      ) : (
        <FlatList
          data={results}
          keyExtractor={(item) => item._id}
          numColumns={2}
          contentContainerStyle={styles.list}
          renderItem={({ item }) => (
            <ProductCard product={item}
              onAddToCart={() => addItem(item)} />
          )}
          ListEmptyComponent={searched ? (
            <Text style={styles.empty}>

```

```

        No se encontraron resultados
      </Text>
    ) : null}
  />
})
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F8F9FA' },
  searchBar: { flexDirection: 'row', alignItems: 'center',
    backgroundColor: '#FFF', margin: 12, padding: 12,
    borderRadius: 10, shadowColor: '#000',
    shadowOpacity: 0.08, shadowRadius: 4,
    elevation: 2 },
  input: { flex: 1, marginLeft: 10, fontSize: 16,
    color: '#2C3E50' },
  loader: { marginTop: 40 },
  list: { padding: 8 },
  empty: { textAlign: 'center', marginTop: 40,
    color: '#95A5A6', fontSize: 15 }
});

```

CONCEPTO APLICADO: Solicitud HTTP Dinámica + Reutilización de Componentes

La pantalla de búsqueda envía GET /products?search=término cada vez que el usuario escribe, demostrando solicitudes HTTP dinámicas. Reutiliza ProductCard del módulo de componentes compartidos, mostrando el principio de reutilización.

11.5. orders.js — Historial de Pedidos

```

// app/(tabs)/orders.js
import { useEffect } from 'react';
import { View, Text, FlatList, ActivityIndicator,
  TouchableOpacity, StyleSheet, Alert } from 'react-native';
import { useOrders } from '../../src/hooks/useOrders';
import { Ionicons } from '@expo/vector-icons';

const STATUS_MAP = {
  pending: { label: 'Pendiente', color: '#F39C12', icon: 'time' },
  confirmed: { label: 'Confirmado', color: '#2E86C1', icon: 'checkmark' },
  shipped: { label: 'Enviado', color: '#8E44AD', icon: 'airplane' },
  delivered: { label: 'Entregado', color: '#148F77', icon: 'checkmark-done' },
  cancelled: { label: 'Cancelado', color: '#E74C3C', icon: 'close' },
};

export default function OrdersScreen() {
  const { orders, loading, loadOrders, cancelOrder } = useOrders();

  useEffect(() => { loadOrders(); }, [loadOrders]);

```

```

if (loading && orders.length === 0) {
  return (
    <View style={styles.center}>
      <ActivityIndicator size="large" color="#1A5276" />
    </View>
  );
}

const handleCancel = (orderId) => {
  Alert.alert('Cancelar Pedido', '¿Estás seguro?',
    [{ text: 'No' },
      { text: 'Sí', style: 'destructive',
        onPress: () => cancelOrder(orderId) }]);
};

const renderOrder = ({ item }) => {
  const status = STATUS_MAP[item.status] || STATUS_MAP.pending;
  return (
    <View style={styles.card}>
      <View style={styles.header}>
        <Text style={styles.orderId}>
          Pedido #{item._id.slice(-6).toUpperCase()}
        </Text>
        <View style={[styles.badge,
          { backgroundColor: status.color }]}>
          <Icons name={status.icon}
            size={14} color="#FFF" />
          <Text style={styles.badgeText}>
            {status.label}
          </Text>
        </View>
      </View>
      <Text style={styles.date}>
        {new Date(item.createdAt).toLocaleDateString('es-PE')}
      </Text>
      <Text style={styles.items}>
        {item.items.length} producto(s)
      </Text>
      <View style={styles.footer}>
        <Text style={styles.total}>
          S/ {item.total.toFixed(2)}
        </Text>
        {item.status === 'pending' && (
          <TouchableOpacity
            onPress={() => handleCancel(item._id)}>
            <Text style={styles.cancelText}>Cancelar</Text>
          </TouchableOpacity>
        )}
      </View>
    </View>
  );
};

return (

```

```

<View style={styles.container}>
  <FlatList
    data={orders}
    keyExtractor={({item} => item._id}
    renderItem={renderOrder}
    contentContainerStyle={styles.list}
    ListEmptyComponent={
      <View style={styles.center}>
        <Ionicons name="receipt-outline" size={48}
          color="#BDC3C7" />
        <Text style={styles.emptyText}>
          No tienes pedidos aún
        </Text>
      </View>
    }
  />
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F8F9FA' },
  center: { flex: 1, justifyContent: 'center',
    alignItems: 'center', paddingTop: 60 },
  list: { padding: 12 },
  card: { backgroundColor: '#FFF', borderRadius: 12,
    padding: 16, marginBottom: 12,
    shadowColor: '#000', shadowOpacity: 0.06,
    shadowRadius: 4, elevation: 2 },
  header: { flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center' },
  orderId: { fontSize: 15, fontWeight: 'bold',
    color: '#1C2833' },
  badge: { flexDirection: 'row', alignItems: 'center',
    paddingHorizontal: 8, paddingVertical: 4,
    borderRadius: 12 },
  badgeText: { color: '#FFF', fontSize: 12,
    fontWeight: '600', marginLeft: 4 },
  date: { fontSize: 13, color: '#7F8C8D', marginTop: 6 },
  items: { fontSize: 13, color: '#566573', marginTop: 2 },
  footer: { flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center', marginTop: 12,
    paddingTop: 12, borderTopWidth: 1,
    borderTopColor: '#ECF0F1' },
  total: { fontSize: 18, fontWeight: 'bold', color: '#1A5276' },
  cancelText: { color: '#E74C3C', fontSize: 14,
    fontWeight: '600' },
  emptyText: { marginTop: 12, color: '#95A5A6', fontSize: 15 }
});

```

CONCEPTO APLICADO: Respuesta JSON + Actualización de UI

OrdersScreen consume GET /orders y renderiza cada pedido con su estado visual (pendiente, confirmado, enviado, entregado, cancelado). El mapeo STATUS_MAP transforma los datos JSON en elementos visuales (colores, iconos, etiquetas). PUT /orders/:id/cancel demuestra actualización del estado.

11.6. checkout.js — Finalizar Compra

```
// app/checkout.js
import { useState } from 'react';
import { View, Text, TextInput, ScrollView,
  TouchableOpacity, Alert, ActivityIndicator,
  StyleSheet } from 'react-native';

import { useRouter } from 'expo-router';
import { useCart } from '../src/hooks/useCart';
import { useOrders } from '../src/hooks/useOrders';
import { Ionicons } from '@expo/vector-icons';

export default function CheckoutScreen() {
  const { items, total, clearCart } = useCart();
  const { createOrder } = useOrders();
  const router = useRouter();
  const [loading, setLoading] = useState(false);
  const [address, setAddress] = useState({
    street: '', city: '', zipCode: ''
  });

  const handlePlaceOrder = async () => {
    if (!address.street || !address.city || !address.zipCode) {
      Alert.alert('Error',
        'Completa todos los campos de dirección');
      return;
    }
    setLoading(true);
    try {
      // POST al endpoint /api/orders
      const orderData = {
        items: items.map(i => ({
          productId: i.productId,
          quantity: i.quantity
        })),
        shippingAddress: address,
        paymentMethod: 'cash'
      };
      await createOrder(orderData);
      await clearCart(); // DELETE /api/cart
      Alert.alert('Pedido Creado',
        'Tu pedido ha sido registrado exitosamente',
        [{ text: 'OK',
          onPress: () => router.replace('/(tabs)/orders') }]);
    } catch (err) {
      Alert.alert('Error', err.message);
    }
  };
}
```



```

    } finally {
        setLoading(false);
    }
};

return (
    <ScrollView style={styles.container}>
        <Text style={styles.sectionTitle}>
            Dirección de Envío
        </Text>
        <TextInput style={styles.input}
            placeholder="Calle y número"
            value={address.street}
            onChangeText={(t) => setAddress({...address, street: t})}
        />
        <TextInput style={styles.input}
            placeholder="Ciudad"
            value={address.city}
            onChangeText={(t) => setAddress({...address, city: t})}
        />
        <TextInput style={styles.input}
            placeholder="Código Postal"
            value={address.zipCode}
            onChangeText={(t) => setAddress({...address, zipCode: t})}
            keyboardType="numeric"
        />

        <Text style={styles.sectionTitle}>Resumen</Text>
        {items.map(item => (
            <View key={item.productId} style={styles.itemRow}>
                <Text style={styles.itemName}>
                    {item.name} x{item.quantity}
                </Text>
                <Text style={styles.itemPrice}>
                    S/ {(item.price * item.quantity).toFixed(2)}
                </Text>
            </View>
        ))}
        <View style={styles.totalRow}>
            <Text style={styles.totalLabel}>Total a Pagar:</Text>
            <Text style={styles.totalAmount}>
                S/ {total.toFixed(2)}
            </Text>
        </View>

        <TouchableOpacity style={styles.placeOrderBtn}
            onPress={handlePlaceOrder} disabled={loading}>
            {loading ? (
                <ActivityIndicator color="#FFF" />
            ) : (
                <>
                    <Ionicons name="checkmark-circle"
                        size={22} color="#FFF" />
                    <Text style={styles.placeOrderText}>

```

```

        Confirmar Pedido
      </Text>
    </>
  )}
</TouchableOpacity>
</ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#FFF', padding: 20 },
  sectionTitle: { fontSize: 18, fontWeight: 'bold',
    color: '#1C2833', marginTop: 16,
    marginBottom: 12 },
  input: { backgroundColor: '#F8F9FA', borderWidth: 1,
    borderColor: '#D5DBDB', borderRadius: 10,
    padding: 14, fontSize: 15, marginBottom: 12 },
  itemRow: { flexDirection: 'row',
    justifyContent: 'space-between', paddingVertical: 8,
    borderBottomWidth: 1, borderBottomColor: '#F2F3F4' },
  itemName: { fontSize: 14, color: '#2C3E50', flex: 1 },
  itemPrice: { fontSize: 14, fontWeight: '600',
    color: '#1A5276' },
  totalRow: { flexDirection: 'row',
    justifyContent: 'space-between', marginTop: 16,
    paddingTop: 16, borderTopWidth: 2,
    borderTopColor: '#1A5276' },
  totalLabel: { fontSize: 18, fontWeight: '600',
    color: '#1C2833' },
  totalAmount: { fontSize: 22, fontWeight: 'bold',
    color: '#1A5276' },
  placeOrderBtn: { flexDirection: 'row',
    backgroundColor: '#148F77', padding: 16,
    borderRadius: 12, justifyContent: 'center',
    alignItems: 'center', marginTop: 30 },
  placeOrderText: { color: '#FFF', fontSize: 18,
    fontWeight: 'bold', marginLeft: 10 }
});

```

CONCEPTO APLICADO: Ciclo Completo: Del Toque al Resultado

CheckoutScreen demuestra el ciclo completo en una sola pantalla: (1) el usuario llena el formulario, (2) toca 'Confirmar Pedido', (3) se envía POST /orders con los items y dirección, (4) el backend valida stock, calcula total, y crea el pedido, (5) DELETE /cart vacía el carrito, (6) la UI navega a la pantalla de pedidos mostrando el resultado.

11.7. profile.js — Perfil de Usuario

```

// app/(tabs)/profile.js
import { View, Text, TouchableOpacity, StyleSheet,
  Alert, ActivityIndicator } from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '../../src/hooks/useAuth';

```

```

import { Ionicons } from '@expo/vector-icons';

export default function ProfileScreen() {
  const { user, loading, logout } = useAuth();
  const router = useRouter();

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#1A5276" />
      </View>
    );
  }

  if (!user) {
    return (
      <View style={styles.center}>
        <Ionicons name="person-circle-outline"
          size={80} color="#BDC3C7" />
        <Text style={styles.guestText}>
          Inicia sesión para ver tu perfil
        </Text>
        <TouchableOpacity style={styles.loginBtn}
          onPress={() => router.push('/auth/login')}>
          <Text style={styles.loginBtnText}>
            Iniciar Sesión
          </Text>
        </TouchableOpacity>
        <TouchableOpacity
          onPress={() => router.push('/auth/register')}>
          <Text style={styles.registerLink}>
            ¿No tienes cuenta? Regístrate
          </Text>
        </TouchableOpacity>
      </View>
    );
  }

  const handleLogout = () => {
    Alert.alert('Cerrar Sesión', '¿Estás seguro?',
      [{ text: 'Cancelar' },
        { text: 'Sí', onPress: logout }]);
  };

  return (
    <View style={styles.container}>
      <View style={styles.header}>
        <View style={styles.avatar}>
          <Text style={styles.avatarText}>
            {user.name.charAt(0).toUpperCase()}
          </Text>
        </View>
        <Text style={styles.userName}>{user.name}</Text>
        <Text style={styles.userEmail}>{user.email}</Text>
      </View>
    </View>
  );
}

```

```

    </View>

    <View style={styles.menu}>
      <MenuItem icon="location" label="Mis Direcciones" />
      <MenuItem icon="heart" label="Favoritos" />
      <MenuItem icon="settings" label="Configuración" />
      <MenuItem icon="help-circle" label="Ayuda" />
    </View>

    <TouchableOpacity style={styles.logoutBtn}
      onPress={handleLogout}>
      <Ionicons name="log-out-outline" size={22}
        color="#E74C3C" />
      <Text style={styles.logoutText}>Cerrar Sesión</Text>
    </TouchableOpacity>
  </View>
);
}

function MenuItem({ icon, label }) {
  return (
    <TouchableOpacity style={styles.menuItem}>
      <Ionicons name={icon} size={22} color="#1A5276" />
      <Text style={styles.menuLabel}>{label}</Text>
      <Ionicons name="chevron-forward" size={20}
        color="#BDC3C7" />
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F8F9FA' },
  center: { flex: 1, justifyContent: 'center',
    alignItems: 'center', padding: 20 },
  guestText: { marginTop: 12, color: '#7F8C8D', fontSize: 15 },
  loginBtn: { backgroundColor: '#1A5276', paddingHorizontal: 40,
    paddingVertical: 14, borderRadius: 10,
    marginTop: 20 },
  loginBtnText: { color: '#FFF', fontSize: 16,
    fontWeight: 'bold' },
  registerLink: { marginTop: 16, color: '#2E86C1',
    fontSize: 14 },
  header: { backgroundColor: '#1A5276', paddingTop: 30,
    paddingBottom: 24, alignItems: 'center' },
  avatar: { width: 70, height: 70, borderRadius: 35,
    backgroundColor: '#148F77',
    justifyContent: 'center', alignItems: 'center' },
  avatarText: { color: '#FFF', fontSize: 28,
    fontWeight: 'bold' },
  userName: { color: '#FFF', fontSize: 20, fontWeight: 'bold',
    marginTop: 10 },
  userEmail: { color: '#D5DBDB', fontSize: 14, marginTop: 4 },
  menu: { marginTop: 16, backgroundColor: '#FFF',
    borderRadius: 12, marginHorizontal: 16 },

```

```
menuItem: { flexDirection: 'row', alignItems: 'center',
            padding: 16, borderBottomWidth: 1,
            borderBottomColor: '#F2F3F4' },
menuLabel: { flex: 1, marginLeft: 14, fontSize: 15,
            color: '#2C3E50' },
logoutBtn: { flexDirection: 'row', alignItems: 'center',
            justifyContent: 'center', marginTop: 30 },
logoutText: { color: '#E74C3C', fontSize: 15,
            fontWeight: '600', marginLeft: 8 }
});
```

CONCEPTO APLICADO: Arquitectura Modular + Consumo de Endpoints

ProfileScreen es un módulo independiente que usa useAuth para verificar si el usuario está autenticado. Si no lo está, muestra botones para login/register (navegación modular). Si sí, muestra los datos del perfil obtenidos via GET /users/profile y opciones de menú como módulos MenuItem reutilizables.

11.8. auth/login.js — Inicio de Sesión

```
// app/auth/login.js
import { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity,
        ActivityIndicator, StyleSheet, Alert } from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '../../src/hooks/useAuth';

export default function LoginScreen() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const { login, loading } = useAuth();
  const router = useRouter();

  const handleLogin = async () => {
    if (!email || !password) {
      Alert.alert('Error', 'Completa todos los campos');
      return;
    }
    try {
      await login(email, password);
      router.replace('/(tabs)/home');
    } catch (err) {
      Alert.alert('Error de autenticación', err.message);
    }
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Bienvenido a NaturApp</Text>
      <Text style={styles.subtitle}>
        Inicia sesión para continuar
      </Text>
    </View>
  );
}
```

```

<TextInput style={styles.input}
  placeholder="Correo electrónico"
  value={email} onChangeText={setEmail}
  keyboardType="email-address" autoCapitalize="none" />

<TextInput style={styles.input}
  placeholder="Contraseña" value={password}
  onChangeText={setPassword} secureTextEntry />

<TouchableOpacity style={styles.button}
  onPress={handleLogin} disabled={loading}>
  {loading ? (
    <ActivityIndicator color="#FFF" />
  ) : (
    <Text style={styles.buttonText}>Iniciar Sesión</Text>
  )}
</TouchableOpacity>

<TouchableOpacity
  onPress={() => router.push('/auth/register')}>
  <Text style={styles.link}>
    ¿No tienes cuenta? Regístrate aquí
  </Text>
</TouchableOpacity>
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#FFF',
    padding: 24, justifyContent: 'center' },
  title: { fontSize: 26, fontWeight: 'bold',
    color: '#1A5276', textAlign: 'center' },
  subtitle: { fontSize: 14, color: '#7F8C8D',
    textAlign: 'center', marginTop: 8,
    marginBottom: 32 },
  input: { backgroundColor: '#F8F9FA', borderWidth: 1,
    borderColor: '#D5DBDB', borderRadius: 10,
    padding: 14, fontSize: 16, marginBottom: 14 },
  button: { backgroundColor: '#1A5276', padding: 16,
    borderRadius: 12, alignItems: 'center',
    marginTop: 10 },
  buttonText: { color: '#FFF', fontSize: 17,
    fontWeight: 'bold' },
  link: { color: '#2E86C1', textAlign: 'center',
    marginTop: 24, fontSize: 14 }
});

```

CONCEPTO APLICADO: Solicitud HTTP POST + Respuesta JSON con Token

LoginScreen envía POST /users/login con email y password. El backend valida las credenciales, genera un token JWT, y devuelve la respuesta JSON { success, data: { user, token } }. El hook

useAuth persiste el token con AsyncStorage y navega al home. Si falla, muestra el mensaje de error del servidor.

12. Frontend — Componentes Reutilizables

Los componentes reutilizables son módulos de UI que se usan en múltiples pantallas. Encapsulan presentación y comportamiento en unidades independientes.

12.1. ProductCard.js — Tarjeta de Producto

```
// src/components/ProductCard.js
import { View, Text, Image, TouchableOpacity,
        StyleSheet } from 'react-native';
import { useRouter } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';

export default function ProductCard({ product, onAddToCart }) {
  const router = useRouter();

  return (
    <TouchableOpacity style={styles.card}
      onPress={() => router.push(`/product/${product._id}`)}
      activeOpacity={0.7}>
      <Image source={{ uri: product.image }}
        style={styles.image}
        defaultSource={require('../assets/placeholder.png')}
      />
      <View style={styles.info}>
        <Text style={styles.category} numberOfLines={1}>
          {product.category?.name}
        </Text>
        <Text style={styles.name} numberOfLines={2}>
          {product.name}
        </Text>
        <View style={styles.row}>
          <Text style={styles.price}>
            S/ {product.price.toFixed(2)}
          </Text>
          <TouchableOpacity style={styles.addBtn}
            onPress={onAddToCart}>
            <Ionicons name="add-circle"
              size={28} color="#148F77" />
          </TouchableOpacity>
        </View>
      </View>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  card: {
    flex: 1, margin: 6, backgroundColor: '#FFF',
    borderRadius: 12,
    shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1, shadowRadius: 4, elevation: 3,
    overflow: 'hidden', maxWidth: '48%'
  }
});
```



```

    },
    image: { width: '100%', height: 140,
              backgroundColor: '#ECF0F1' },
    info: { padding: 10 },
    category: { fontSize: 11, color: '#148F77',
                textTransform: 'uppercase', fontWeight: '600' },
    name: { fontSize: 14, fontWeight: 'bold', color: '#2C3E50',
            marginTop: 4 },
    row: { flexDirection: 'row', justifyContent: 'space-between',
           alignItems: 'center', marginTop: 8 },
    price: { fontSize: 16, fontWeight: 'bold', color: '#1A5276' },
    addBtn: { padding: 4 }
  });

```

CONCEPTO APLICADO: Reutilización + Modularidad

ProductCard se usa tanto en HomeScreen como en SearchScreen, demostrando la reutilización de módulos. Recibe props (product, onAddToCart) y encapsula la navegación al detalle con router.push(), manteniendo su lógica independiente de la pantalla que lo contiene.

12.2. CartItemRow.js — Fila de Item en Carrito

```

// src/components/CartItemRow.js
import { View, Text, Image, TouchableOpacity,
         StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';

export default function CartItemRow({ item, onUpdateQty,
                                     onRemove }) {

  return (
    <View style={styles.row}>
      <Image source={{ uri: item.image }}
        style={styles.image} />
      <View style={styles.info}>
        <Text style={styles.name} numberOfLines={2}>
          {item.name}
        </Text>
        <Text style={styles.price}>
          S/ {item.price.toFixed(2)}
        </Text>
      <View style={styles.qtyRow}>
        <TouchableOpacity
          onPress={() => onUpdateQty(item.quantity - 1)}
          style={styles.qtyBtn}>
          <Ionicons name="remove-circle-outline"
            size={24} color="#1A5276" />
        </TouchableOpacity>
        <Text style={styles.qty}>{item.quantity}</Text>
        <TouchableOpacity
          onPress={() => onUpdateQty(item.quantity + 1)}
          style={styles.qtyBtn}>
          <Ionicons name="add-circle-outline"
            size={24} color="#1A5276" />
        </TouchableOpacity>
      </View>
    </View>
  );
}

```

```

        </TouchableOpacity>
      </View>
    </View>
    <View style={styles.right}>
      <Text style={styles.subtotal}>
        S/ {(item.price * item.quantity).toFixed(2)}
      </Text>
      <TouchableOpacity onPress={onRemove}
        style={styles.removeBtn}>
        <Icons name="trash-outline"
          size={20} color="#E74C3C" />
      </TouchableOpacity>
    </View>
  </View>
);
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', backgroundColor: '#FFF',
    borderRadius: 10, padding: 12, marginBottom: 10,
    shadowColor: '#000', shadowOpacity: 0.05,
    shadowRadius: 4, elevation: 2 },
  image: { width: 70, height: 70, borderRadius: 8,
    backgroundColor: '#ECF0F1' },
  info: { flex: 1, marginLeft: 12 },
  name: { fontSize: 14, fontWeight: '600', color: '#2C3E50' },
  price: { fontSize: 13, color: '#7F8C8D', marginTop: 2 },
  qtyRow: { flexDirection: 'row', alignItems: 'center',
    marginTop: 6 },
  qtyBtn: { padding: 2 },
  qty: { fontSize: 16, fontWeight: 'bold', marginHorizontal: 12,
    color: '#1C2833' },
  right: { alignItems: 'flex-end',
    justifyContent: 'space-between' },
  subtotal: { fontSize: 15, fontWeight: 'bold',
    color: '#1A5276' },
  removeBtn: { marginTop: 8 }
});

```

CONCEPTO APLICADO: Modularidad + Mantenimiento

CartItemRow encapsula la presentación y controles de un ítem del carrito como módulo independiente. Si se necesita cambiar el diseño de la fila o agregar funcionalidad (ej. swipe-to-delete), solo se modifica este archivo sin afectar a CartScreen.

12.3. CategoryChips.js — Chips de Categorías

```

// src/components/CategoryChips.js
import { ScrollView, TouchableOpacity, Text,
  StyleSheet } from 'react-native';
import { Icons } from '@expo/vector-icons';

const ICONS = { 'Superalimentos': 'nutrition', 'Infusiones': 'cafe',

```

```

'Suplementos': 'fitness', 'Cuidado Personal': 'flower',
'Snacks': 'fast-food' });

export default function CategoryChips({ categories, selected,
                                     onSelect }) {
  return (
    <ScrollView horizontal showsHorizontalScrollIndicator={false}
      style={styles.container}
      contentContainerStyle={styles.content}>
      <TouchableOpacity
        style={[styles.chip, !selected && styles.chipActive]}
        onPress={() => onSelect(null)}>
        <Ionicons name="grid" size={16}
          color={!selected ? '#FFF' : '#1A5276'} />
        <Text style={[styles.chipText,
          !selected && styles.chipTextActive]}>Todos</Text>
      </TouchableOpacity>

      {categories.map((cat) => (
        <TouchableOpacity key={cat._id}
          style={[styles.chip,
            selected === cat._id && styles.chipActive]}
          onPress={() => onSelect(cat._id)}>
          <Ionicons
            name=ICONS[cat.name] || 'leaf'
            size={16}
            color={selected === cat._id ? '#FFF' : '#1A5276'} />
          <Text style={[styles.chipText,
            selected === cat._id && styles.chipTextActive]}>
            {cat.name}
          </Text>
        </TouchableOpacity>
      ))}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { maxHeight: 52, backgroundColor: '#FFF',
    borderBottomWidth: 1,
    borderBottomColor: '#ECF0F1' },
  content: { paddingHorizontal: 12, paddingVertical: 8,
    alignItems: 'center' },
  chip: { flexDirection: 'row', alignItems: 'center',
    paddingHorizontal: 14, paddingVertical: 8,
    borderRadius: 20, backgroundColor: '#EBF5FB',
    marginRight: 8 },
  chipActive: { backgroundColor: '#1A5276' },
  chipText: { marginLeft: 6, fontSize: 13, fontWeight: '600',
    color: '#1A5276' },
  chipTextActive: { color: '#FFF' }
});

```

CONCEPTO APLICADO: Reutilización + Consumo de Datos

CategoryChips recibe las categorías obtenidas del endpoint GET /categories y las renderiza como filtros horizontales. El patrón de componente controlado (selected + onSelect) permite reutilizarlo en cualquier pantalla que necesite filtrado por categoría.

13. Flujo de Datos Completo

A continuación se describe el flujo de datos completo de NaturApp, desde que el usuario toca un elemento hasta que ve el resultado en pantalla. Este flujo ilustra las **5 fases del ciclo de comunicación** presentadas en la Sesión 11:

Fase del Flujo	Descripción en NaturApp	Archivos Involucrados
Fase 1: Inicio de Solicitud (0-50ms)	El usuario toca 'Agregar al Carrito' en ProductCard. El evento onPress dispara addItem(product) del hook useCart, que a su vez llama a CartAPI.addItem() en apiService.js.	ProductCard.js, useCart.js
Fase 2: Viaje a través de la Red (50-200ms)	La función request() de apiService.js construye la solicitud HTTP: POST a /api/cart/add con el body JSON serializado y el header Authorization: Bearer <token>. fetch() envía la solicitud a través de la red al servidor.	apiService.js (request)
Fase 3: Procesamiento del Servidor (200-500ms)	Express.js recibe la solicitud en cartRoutes.js. El middleware authenticate verifica el token JWT. El handler busca si el producto ya existe en el carrito; si existe, incrementa la cantidad; si no, lo agrega. El servidor responde con el carrito actualizado.	auth.js, cartRoutes.js
Fase 4: Retorno de Datos (500-700ms)	La respuesta JSON { success: true, data: { items, total, count } } viaja de vuelta al dispositivo móvil. apiService.js verifica response.ok y deserializa el JSON.	apiService.js (request)
Fase 5: Renderizado de UI (700-1000ms)	useCart ejecuta loadCart() que actualiza el estado (setItems, setTotal, setCount) con los nuevos datos. React Native detecta el cambio de estado y re-renderiza automáticamente CartScreen mostrando el nuevo item.	useCart.js, cart.js

CONCEPTO APLICADO: Flujo de Datos en Tiempo Real + Consideraciones de Performance

NaturApp implementa las técnicas de performance mencionadas en la Sesión 11: (1) Spinners (ActivityIndicator) en todas las pantallas durante la carga; (2) Solicitudes en paralelo (Promise.all) en useProducts para cargar productos y categorías simultáneamente; (3) Paginación infinita (onEndReached) para no cargar todos los productos a la vez; (4) Pull-to-refresh (RefreshControl) para actualizar datos sin recargar la app; (5) Estados de error con mensajes descriptivos del servidor.

14. Tabla Resumen de Conceptos Aplicados

La siguiente tabla resume todos los conceptos de la Sesión 11 y su implementación verificable en el código:

Concepto	Implementación	Aplicado
Arquitectura Modular	Estructura de carpetas con módulos independientes	Sí
Modularidad	Cada archivo tiene responsabilidad única	Sí
Reutilización	ProductCard, apiService, middleware auth	Sí
Mantenimiento	Cambios aislados por módulo	Sí
Backend con Endpoints	Express.js con 5 grupos de rutas	Sí
RESTful APIs (GET/POST/PUT/DELETE)	CRUD completo en cada módulo de rutas	Sí
Endpoints con URL única	/api/products, /api/orders, etc.	Sí
Arquitectura Cliente-Servidor	React Native (cliente) + Express (servidor)	Sí
Librerías de Red (fetch)	apiService.js con fetch nativo	Sí
Solicitud HTTP configurada	Headers, Authorization, body JSON	Sí
Procesamiento Backend	Validación, lógica de negocio, MongoDB	Sí
Respuesta JSON estructurada	{ success, data, message } + códigos HTTP	Sí
Actualización de UI	useState/useEffect + re-renderizado	Sí
Flujo de Datos (5 fases)	Toque → HTTP → Servidor → JSON → UI	Sí
Performance: Spinners	ActivityIndicator en todas las pantallas	Sí
Performance: Solicitudes paralelo	Promise.all en useProducts	Sí
Performance: Paginación	onEndReached + loadMore	Sí
Performance: Pull-to-refresh	RefreshControl en HomeScreen	Sí
Ciclo Completo	3 pilares integrados en la app funcional	Sí